

Instituto Tecnológico de Costa Rica

Defensa y Asalto de Base

Documentación del Proyecto

Estudiante: Nazareth Ulate Diaz
Curso: Introducción a la Programación
Modalidad: Live Learning
Carrera: Ingeniería en Computación
Fecha de entrega: 19 de junio de 2026

Repositorio: <https://github.com/nazth703/defensa-y-asalto-base>

1. Descripcion General del Juego

Defensa y Asalto de Base es un juego de estrategia por turnos para dos jugadores. Un jugador asume el rol de defensor y debe construir una base con muros y torres defensivas, mientras que el otro jugador asume el rol de atacante y debe destruir esa base utilizando diferentes tipos de unidades. El juego se desarrolla por rondas y el primer jugador que gane 3 rondas gana la partida completa.

Antes de iniciar, ambos jugadores deben registrarse o iniciar sesion, elegir una faccion distinta (Medieval, Naturaleza u Oscura) y seleccionar su rol para la partida.

2. Reglas del Juego

Desarrollo de una ronda:

- El defensor recibe dinero inicial y coloca torres y muros en el mapa.
- El atacante recibe dinero inicial y coloca sus unidades en las columnas de la derecha.
- Se ejecuta la fase de combate automaticamente.
- Las torres atacan a las unidades dentro de su alcance.
- Las unidades avanzan hacia la base y atacan muros, torres o la base central.
- Se determina el ganador de la ronda.
- Si ningun jugador ha ganado 3 rondas, inicia una nueva ronda.

Condiciones de victoria:

El defensor gana la ronda si:

- Todas las unidades atacantes son eliminadas.
- La base central no fue destruida.

El atacante gana la ronda si:

- Logra destruir la base central del defensor.

El primer jugador que gane 3 rondas gana la partida completa. Al finalizar, se actualiza el registro de victorias segun el rol jugado.

Sistema de dinero:

- Ambos jugadores inician cada ronda con \$200.
- El defensor gana dinero por cada unidad eliminada.
- El atacante gana dinero por dano realizado a torres y a la base.
- Al inicio de cada nueva ronda, ambos jugadores reciben \$150 adicionales.
- Click derecho sobre un elemento lo retira y devuelve el dinero completo.

3. Torres Defensivas

El defensor puede construir tres tipos de torres. Cada una tiene nombre, costo, vida, dano, alcance y una habilidad especial que se activa despues de cierta cantidad de turnos.

Torre	Costo	Vida	Dano	Alcance	Habilidad Especial	Turnos
Torre Basica	\$50	100	20	3	Disparo doble: ataca dos veces a la unidad mas cercana.	3
Torre Pesada	\$120	300	50	2	Dano en area: daña a todas las unidades en su alcance.	4

Torre Magica	\$80	80	15	4	Congela: inmoviliza una unidad por 2 turnos.	3
--------------	------	----	----	---	--	---

Nota: los costos y estadísticas pueden variar ligeramente según la facción elegida.

4. Unidades Atacantes

El atacante puede comprar tres tipos de unidades. Cada una tiene nombre, costo, vida, dano, velocidad (casillas por turno) y una habilidad especial.

Unidad	Costo	Vida	Dano	Velocidad	Habilidad Especial	Turnos
Soldado	\$30	80	15	1	Ataque doble: golpea dos veces al objetivo mas cercano.	3
Tanque	\$100	300	30	1	Escudo temporal: reduce el dano recibido a la mitad.	4
Unidad Rapida	\$50	60	10	2	Aumento de velocidad: se mueve 3 casillas en lugar de 2.	2

5. Facciones

Cada jugador debe elegir una facción distinta antes de jugar. Las facciones modifican el aspecto visual del mapa, torres, muros y unidades, y pueden incluir pequeñas variaciones en las estadísticas de forma balanceada.

Facción	Descripción	Color en el mapa
Medieval	Muros de piedra y torres de arqueros. Fuertes y resistentes.	Tonos café y marrón
Naturaleza	Trampas de raíces y torres de druidas. Veloces y ágiles.	Tonos verdes
Oscura	Muros de obsidiana y torres de sombras. Alta resistencia.	Tonos morados y oscuros

6. Capturas de Pantalla

Defensa y Asalto de Base

DEFENSA Y ASALTO DE BASE

Inicia sesión o regístrate para jugar

[Ver Ranking](#)

Música: ON

 **Jugador 1**
= No conectado

USUARIO

CONTRASEÑA

[Iniciar Sesión](#) [Registrarse](#)

 **Jugador 2**
= No conectado

USUARIO

CONTRASEÑA

[Iniciar Sesión](#) [Registrarse](#)

[INICIAR PARTIDA](#)

Pantalla de inicio de sesion y registro de jugadores.

Selección de Facciones

Elige tu Facción

naza

 **Medieval**
Muros de piedra y torres de arqueros.
Fuertes y resistentes.

 **Naturaleza**
Trampas de raíces y torres de druidas.
Veloces y ágiles.

 **Oscura**
Muros de obsidiana y torres de sombras.
Alta resistencia.

Ari

 **Medieval**
Muros de piedra y torres de arqueros.
Fuertes y resistentes.

 **Naturaleza**
Trampas de raíces y torres de druidas.
Veloces y ágiles.

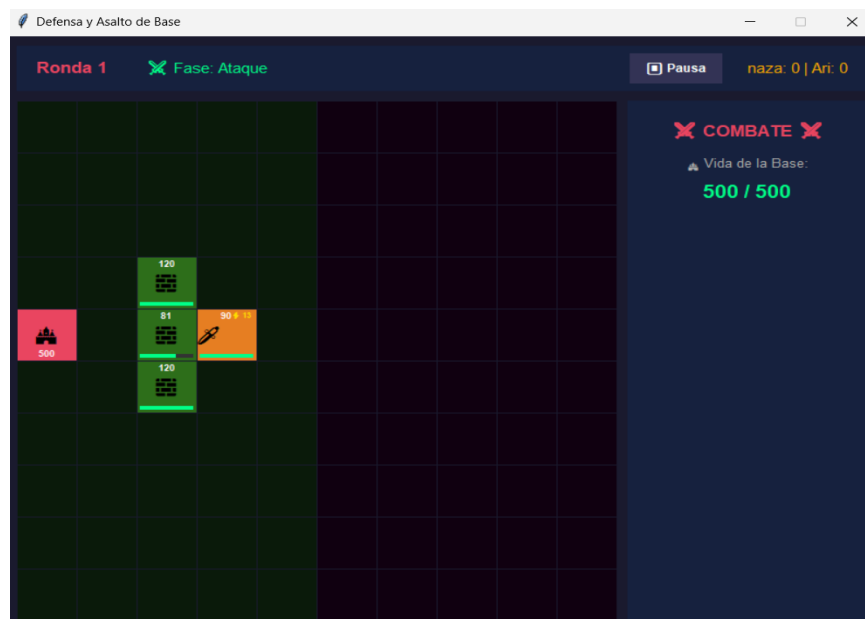
 **Oscura**
Muros de obsidiana y torres de sombras.
Alta resistencia.

[Confirmar y Continuar](#)

Pantalla de seleccion de facciones para cada jugador.



Pantalla de seleccion de roles (defensor o atacante).



Mapa del juego durante la fase de combate.

7. Estructuras de Datos Utilizadas

Lista de listas (Matriz 2D): El mapa del juego se representa como una matriz de 10x10 donde cada celda puede contener None (vacío), una Torre, un Muro, una Unidad o la BaseCentral.

Listas: Se usan listas para almacenar las torres activas, muros activos y unidades en combate. Se filtran en cada turno para eliminar los elementos destruidos.

Diccionarios (JSON): Los datos de jugadores se almacenan en un archivo JSON como una lista de diccionarios con campos: usuario, contraseña, victorias_defensor y victorias_atacante.

Herencia de clases: Se usa herencia para modelar los tipos de torres (TorreBasica, TorrePesada, TorreMagica) y unidades (Soldado, Tanque, UnidadRapida), todas con una clase base comun.

8. Sistema de Archivos

El proyecto utiliza un archivo JSON para persistir los datos de los jugadores entre sesiones. El archivo se encuentra en datos/jugadores.json y es gestionado por el modulo clases/archivo_jugadores.py.

- inicializar_archivo(): crea el archivo si no existe al iniciar el programa.
- registrar_jugador(): agrega un nuevo jugador verificando que no exista previamente.
- iniciar_sesion(): valida usuario y contrasena contra los datos guardados.
- actualizar_victorias(): suma una victoria al jugador segun el rol que jugo.
- obtener_top_defensores() y obtener_top_atacantes(): retornan los 5 mejores jugadores de cada categoria.

Estructura del archivo jugadores.json:

```
[ { "usuario": "naza", "contrasena": "1234", "victorias_defensor": 2, "victorias_atacante": 0 } ]
```

9. Instrucciones para Ejecutar el Programa

1. Tener Python 3.10 o superior instalado.
2. Clonar o descargar el repositorio del proyecto.
3. Colocar un archivo musica.wav en la carpeta assets/sonidos/ (opcional).
4. Ejecutar el archivo principal desde la terminal con: python main.py
5. No ejecutar archivos individuales como mapa.py o login.py directamente.

10. Conclusiones

Nazareth Ulate Diaz

Durante el desarrollo de este proyecto aprendi a estructurar un programa complejo utilizando programacion orientada a objetos, dividiendo la logica en clases independientes que se comunican entre si. Uno de los mayores retos fue implementar el sistema de combate automatico, especialmente lograr que las unidades avanzaran correctamente por el mapa, atacaran la base y que los datos de vida se actualizaran visualmente en tiempo real. Tambien fue desafiante conectar todas las ventanas del juego de forma ordenada mediante callbacks, y resolver errores de compatibilidad con las bibliotecas de sonido. Ver el juego funcionar completamente, con las barras de vida, los numeros de dano flotantes y la musica de fondo, fue muy satisfactorio y me demostro que con organizacion y persistencia es posible construir un proyecto de esta magnitud desde cero.

11. Links del Proyecto

Repositorio GitHub: <https://github.com/nazth703/defensa-y-asalto-base>

Video evidencia: <https://youtu.be/D2mCfz7sLI8?si=fpMRPxi19goZ7UYd>